ASE CHAPTER 1

T3 · Understand (Why / How)

Explain-level questions grounded in Aggarwal — answers in English, traps and notes in Arabic

Chapter 1 — Introduction

كيف تستخدم الملف. اقرأ السؤال، جاوب بكلماتك، بعدين افتح البطاقة قارن. **الجواب بالإنجليزي** — هذا اللي تترجمه بالورقة. **البطاقات الصفراء والحمراء بالعربي** — فخاخ وتنبيهات.

المصدر. Aggarwal & Singh, *Software Engineering*, 3rd ed. (الطبعة الثالثة) — الكتاب الممسوح AliFahim_Software_Engineering.pdf. النص = ص.1–16 · مفتاح الأجوبة = **ص.498**. أغروال هو المصدر الأساسي؛ أي كتاب ثاني نوضّح مصدره.

طريقة قراءة الإجابات

الكلام العريض مبني على المصدر، وقد يُبسّط بألفاظ أسهل. الكلام العادي صياغة أو استنتاج مختصر. الاقتباس الحرفي فقط يوضع بين علامتي تنصيص.

Retrieval set — T3 Understand (why / how)

U.1 · Why has the main goal of software development moved from simply producing good software to producing software that is also maintainable?

Answer

Because the cost of maintaining software increased, the objective is now to produce software that is maintainable, delivered on time, within budget, and that satisfies its requirements.

Aggarwal, §1.2.1, p.4.

U.2 · Why is the software crisis said to be made worse by the progress of hardware technology? Explain with examples.

Answer

Rapid hardware gains contrasted with slower software progress because software faces difficulties such as complexity, changeability, and invisibility. Y2K and Ariane-5 illustrate software failures; they do not by themselves prove that hardware progress caused the crisis.

Two examples from the failure list: **Y2K involved shortening the four-digit year to a two-digit format, such as 64; Ariane-5 suffered an overflow when the guidance computer converted sideways velocity from a 64-bit format to a 16-bit format.**

Aggarwal, §1.1.2, p.3; §1.4, p.12; §1.1.1, pp.2–3.

فخ هذا السؤال

لا تخلط بين تزامن تقدّم العتاد ومشكلات البرمجيات وبين إثبات علاقة سببية مباشرة. تغيّر نسبة كلفة العتاد إلى البرمجيات عامل مستقل يورده الكتاب ضمن أسباب الأزمة.

عبارة «practically non-existent» **تنطبق على Y2K فقط** — الكتاب يسمّي مشكلة Y2K نفسها كذلك، والضرر الحقيقي كان كلفة معالجة المشكلة. أما **Ariane 501 فكان فشلاً فعلياً** — خسِر فيه الصاروخ بسبب overflow التحويل من 64-bit إلى 16-bit — فلا تُعمّم وصف Y2K عليه.

U.3 · How do software myths interfere with the way software should be developed? Give the myths and their effects.

Answer

"There are number of myths associated with software development community. Some of them really affect the way, in which software development should take place." Nine myths and how each warps the process:

1. Software is easy to change. "Every change requires that the complete system be re-verified."

2. Computers provide greater reliability than the devices they replace. "Now, we have software error as well."

3. Testing or 'proving' software correct can remove all the errors. "Testing can only show the presence of errors. It cannot show the absence of errors."

4. Reusing software increases safety. "A false sense of security."

5. Software can work right the first time. "No aeronautical engineer would build a jet fighter without a prototype."

6. Software can be designed thoroughly enough to avoid most integration problems. "There is an old saying among software designers: 'Too bad, there is no compiler for specifications'."

7. Software with more features is better software. "The best, most enduring programs are those which do one thing well."

8. Addition of more software engineers will make up the delay. It **"may further delay the project."**

9. Aim is to develop working programs. "The aim has been shifted from developing working programs to good quality, maintainable programs."

Aggarwal, §1.4, pp.11–12.

U.4 · What is the software process, and why is it so hard to improve?

Answer

"The software process is the way in which we produce software."

Four reasons it is difficult to improve [HUMP89, WIEG99]: **not enough time** — unrealistic schedules leave no room to study what is wrong · **lack of knowledge** — developers do not read the literature on best-known practice · **wrong motivations** — initiatives launched to satisfy a contractor or a manager's CMM ambition · **insufficient commitment** — assessment without follow-through, no plan and no roadmap.

Add Fig. 1.4: improvement **"will not have an impact on current productivity"** — the curve dips before it rises.

Aggarwal, §1.2.3, pp.6–7; Fig. 1.4, p.7.

U.5 · What is the difference between a software metric and a software measurement? Why does the distinction matter?

Answer

Metric: "a quantitative measure of the degree to which a system, component, or process possesses a given attribute." Measurement: "the act of evaluating a measure." One is the quantity a system possesses; the other is the act of evaluating it.

Pressman [PRES02]: **"When a single data point has been collected (e.g., the number of errors uncovered in the review of a single module), a measure has been established. Measurement occurs as the result of the collection of one or more data points... A software metric relates the individual measures in some way (e.g., the average number of errors found per review)."**

Aggarwal, §1.5.3, p.13.

تدقيق المطالبات (claim check) — هذا الملف

الكلام العريض يلخّص المادة المسندة للمصدر، والنص العادي يوضح الصياغة أو الاستنتاج. المراجع مثبتة تحت كل جواب.

Rapid self-test — recall prompts (no answers shown)

Why did the goal shift from "working programs" to "maintainable programs"?
Give two software-failure examples from the chapter, and explain whether they establish a direct link to hardware progress.
State any three of the nine software myths and their effect on the process.
Give the four reasons software process improvement is difficult.
Define metric, measurement, and measure in one line each.